Методические рекомендации. Классификация

1 Оценка моделей
- 1.1 Оценивание предсказаний бинарного классификатора
- 1.2 Оценивание порогов бинарного классификатора
- 1.3 Оценивание предсказаний мультиклассового классификатора
- 1.4 Визуализация результативности классификатора

2 Деревья решений
- 2.1 Обучение дерева решений для задач классификации
- 2.2 Управление размером дерева
- 2.3 Преимущества и недостатки дерева решений

3 K ближайших соседей
- 3.1 Поиск ближайших соседей наблюдения
- 3.2 Создание классификационной модели k ближайших соседей
- 3.3 Идентификация наилучшего размера окрестности
- 3.3 Преимущества и недостатки k ближайших соседей

4 Логистическая регрессия
- 4.1 Логистическая регрессия для бинарной классификации
- 4.2 Обучение мультиклассового классификатора
- 4.3 Снижение дисперсии с помощью регуляризации
- 4.4 Обработка несбалансированных классов
- 4.5 Преимущества и недостатки логистической регрессии

5 Наивный байесовский классификатор
- 5.1 Тренировка классификатора для непрерывных признаков
- 5.2 Калибровка предсказанных вероятностей наивного байесовского классификатора
- 5.3 Преимущества и недостатки наивного байесовского классификатора

1 Оценка моделей

1.1 Оценивание предсказаний бинарного классификатора

Оценка качества классификатора — это способ понять, насколько хорошо модель справляется с разделением объектов на два класса. Метрики

помогают определить, насколько точны и надёжны её ответы. Без таких показателей невозможно понять, работает ли модель правильно, нужно ли её доработать или стоит выбрать другой алгоритм.

В задачах классификации важно учитывать не только общее количество правильных ответов, но и то, какие именно ошибки делает модель — например, как часто она путает один класс с другим. Для этого применяются разные метрики: accuracy, precision, recall, *F_1-мера*.

Выбор подходящих метрик помогает лучше понять, как ведёт себя классификатор, и правильно оценить его качество в зависимости от особенностей данных и цели исследования.

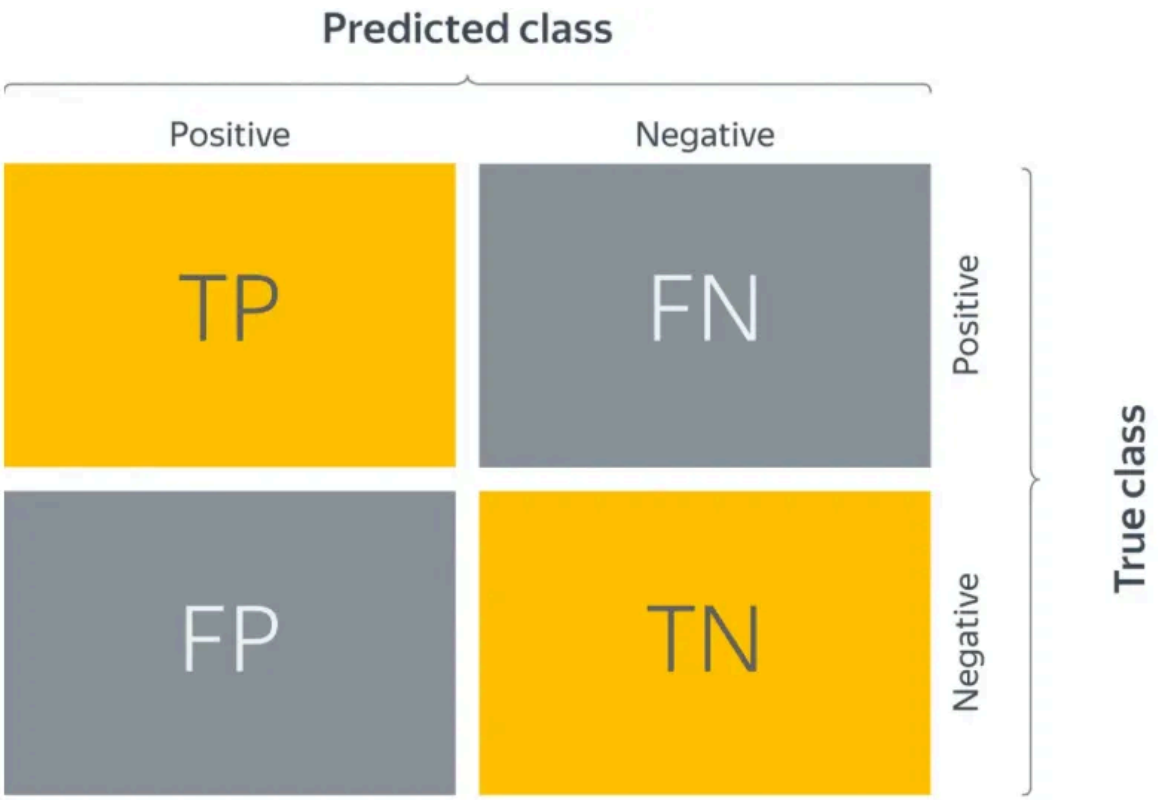
Accuracy

Accuracy — это наиболее распространённая метрика, показывающая долю правильных предсказаний модели.

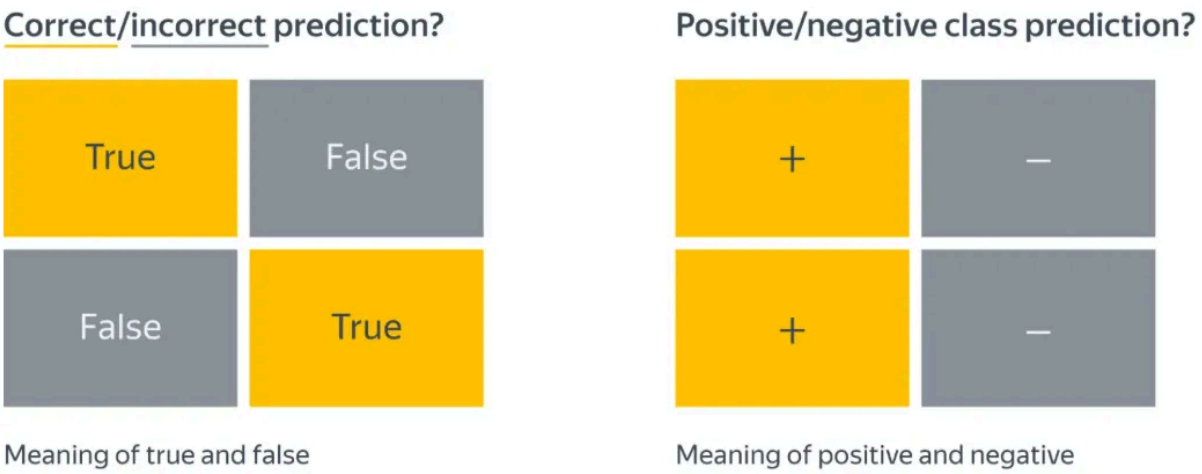
Формула:

где:

- **TP (True Positive)** — количество истинно положительных исходов (например, модель правильно предсказала, что человек болен);
- **TN (True Negative)** — количество истинно отрицательных исходов (модель правильно предсказала, что человек здоров);
- **FP (False Positive)** — количество ложноположительных исходов (ошибка I рода, когда модель ошибочно предсказывает положительный класс);
- **FN (False Negative)** — количество ложноотрицательных исходов (ошибка II рода, когда модель не распознаёт положительный класс).



Не волнуйтесь, если первое время эти обозначения будут сводить вас с ума (будем откровенны, даже профи со стажем в них порой путаются), однако логика за ними достаточно простая: первая часть названия группы показывает угадали ли мы с классом, а вторая — какой класс мы предсказали.



Пример вычисления accuracy:

```
# Загрузить библиотеки
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification

# Сгенерировать данные
X, y = make_classification(n_samples=10000,
                           n_features=3,
                           n_informative=3,
                           n_redundant=0,
                           n_classes=2,
                           random_state=1)

# Создать модель логистической регрессии
logit = LogisticRegression()

# Провести перекрёстную проверку с метрикой точности
cross_val_score(logit, X, y, scoring="accuracy")
# Результат:
# array([0.95170966, 0.9580084 , 0.95558223])
```

Метрика **accuracy** проста и интуитивно понятна — это доля верно классифицированных наблюдений. Однако она может вводить в заблуждение, если классы в данных сильно несбалансированы.

Например, если 99,9% объектов принадлежат к одному классу, модель может просто всегда предсказывать этот класс и получать высокую точность, не имея реальной предсказательной способности.

Точность (Precision)

Точность отражает, насколько модель “точна” в своих положительных предсказаниях — то есть, долю истинно положительных наблюдений среди всех предсказанных как положительные:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Модели с высокой точность делают меньше ложных тревог, но могут пропускать положительные случаи (являются “пессимистичными”).

```
cross_val_score(logit, X, y, scoring="precision")
# array([0.95252404, 0.96583282, 0.95558223])
```

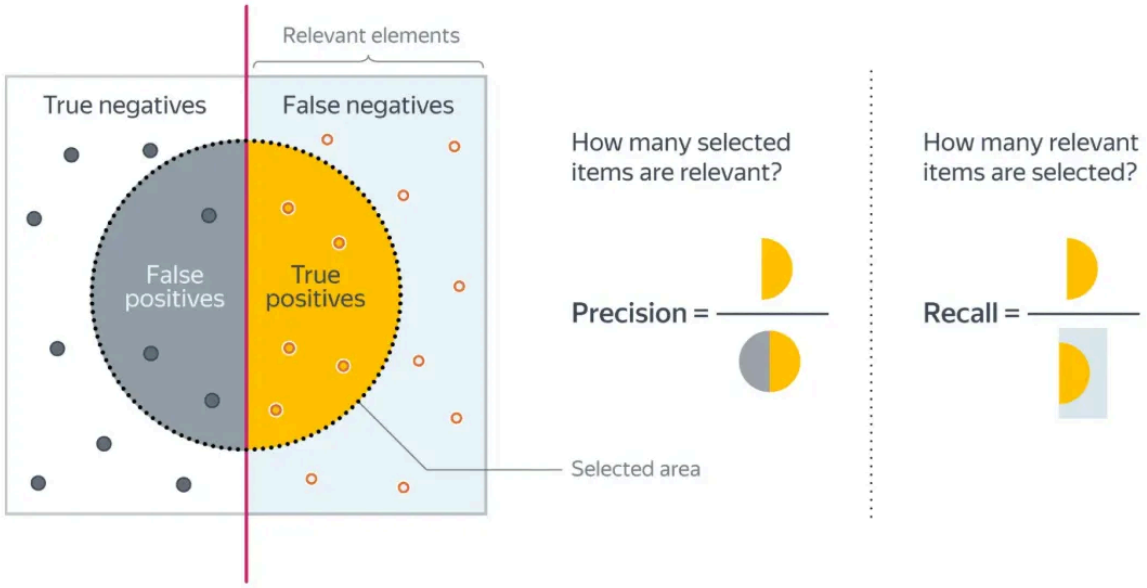
Полнота (Recall)

Полнота показывает, насколько хорошо модель находит все положительные объекты — то есть, долю истинно положительных случаев среди всех реальных положительных наблюдений:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Модели с высокой полнотой чаще классифицируют наблюдения как положительные, рискуя при этом сделать больше ложных срабатываний (являются “оптимистичными”).

```
cross_val_score(logit, X, y, scoring="recall")
# array([0.95080984, 0.94961008, 0.95558223])
```



Оценка F_1 (F_1-score)

F_1-оценка объединяет точность и полноту, находя их гармоническое среднее. Это удобная метрика, когда важно сбалансировать обе характеристики:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
cross_val_score(logit, X, y, scoring="f1")
# array([0.95166617, 0.95765275, 0.95558223])
```

F_1-оценка помогает понять, насколько сбалансированы “оптимизм” (полнота) и “пессимизм” (точность) модели.

Она особенно полезна при работе с несбалансированными выборками, где accuracy не отражает реального качества модели.

Вычисление метрик без кросс-валидации

Если у нас уже есть реальные значения `y` и предсказания `y_hat`, можно вычислить метрики напрямую:

```
# Загрузить библиотеки
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Разбить данные
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_s
tate=1)

# Обучить модель и сделать предсказания
y_hat = logit.fit(X_train, y_train).predict(X_test)

# Вычислить точность
accuracy_score(y_test, y_hat)
# 0.947
```

Accuracy — полезная и интуитивная метрика, но она может быть обманчивой при несбалансированных данных.

В таких случаях лучше использовать комбинацию метрик: точность, полноту и F_1-оценку, которые дают более точное представление о способности модели различать классы и делать обоснованные предсказания.

1.2 Оценивание порогов бинарного классификатора

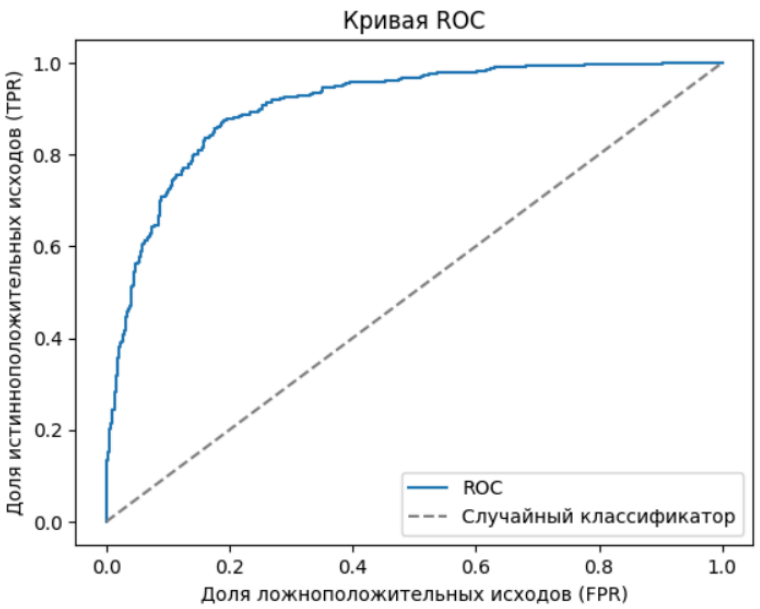
В бинарной классификации модель может выдать не только предсказанный класс, но и вероятность принадлежности объекта к положительному классу. Чтобы принять окончательное решение, необходимо задать **пороговое значение** — границу, после которой модель считает объект положительным. От выбора этого порога напрямую зависит соотношение правильных и ошибочных предсказаний.

Изменение порога может существенно повлиять на результаты: при слишком низком пороге модель склонна выдавать больше ложных срабатываний, а при слишком высоком — пропускать истинно положительные случаи. Поэтому важно уметь оценивать, как выбор порога влияет на поведение классификатора и находить оптимальный баланс между чувствительностью и специфичностью.

Для такого анализа используют **ROC-кривую (Receiver Operating Characteristic curve)**, которая показывает зависимость между долей истинноположительных (True Positive Rate, TPR) и ложноположительных (False Positive Rate, FPR) предсказаний при разных порогах. По форме ROC-кривой можно судить о способности модели различать классы: чем ближе кривая к верхнему левому углу, тем лучше работает классификатор.

```
# Загрузить библиотеки
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.model_selection import train_test_split

# Создать матрицу признаков и вектор целей
features, target = make_classification(
    n_samples=10000,
    n_features=10,
    n_classes=2,
    n_informative=3,
    random_state=3
)
```

```
# Разбить данные на тренировочный и тестовый наборы
features_train, features_test, target_train, target_test = train_test_split(
    features, target, test_size=0.1, random_state=1
)

# Создать логистический регрессионный классификатор
logit = LogisticRegression()

# Натренировать модель
logit.fit(features_train, target_train)

# Получить предсказанные вероятности для положительного класса
target_probabilities = logit.predict_proba(features_test)[:, 1]

# Создать доли истинно- и ложноположительных исходов
false_positive_rate, true_positive_rate, threshold = roc_curve(target_test, ta
rget_probabilities)

# Построить график ROC-кривой
plt.title("Кривая ROC")
plt.plot(false_positive_rate, true_positive_rate, label="ROC")
plt.plot([0, 1], [0, 1], "--", color="gray", label="Случайный классификато
р")
plt.xlabel("Доля ложноположительных исходов (FPR)")
plt.ylabel("Доля истинноположительных исходов (TPR)")
plt.legend()
plt.show()
```



Ранее мы оценивали модели на основе дискретных предсказаний (0 или 1).

Однако большинство алгоритмов машинного обучения, включая **логистическую регрессию**, выдают **вероятности принадлежности** к каждому классу.

Например, если вызвать метод `predict_proba`, можно увидеть вероятностное распределение:

```
# Получить предсказанные вероятности
logit.predict_proba(features_test)[0:1]
```

```
array([[0.8688938, 0.1311062]]
```

Атрибут `classes_` показывает порядок классов:

```
logit.classes_
```

```
array([0, 1])
```

В данном примере первое наблюдение имеет **87% вероятность принадлежности к отрицательному классу (0)** и **13% — к положительному (1)**.

По умолчанию scikit-learn относит наблюдение к положительному классу, если вероятность превышает **0.5**.

Тем не менее, этот **порог можно изменить** в зависимости от контекста задачи.

Например, если **ложноположительная ошибка** (False Positive) обходится компании слишком дорого (например, в банковском скоринге), стоит повысить порог вероятности.

В этом случае модель будет делать меньше ложных тревог, но может пропустить часть действительно положительных наблюдений.

Такой компромисс отражается на двух показателях

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

Пример влияния порога

```
print("Порог:", threshold[116])
print("Доля истинноположительных:", true_positive_rate[116])
print("Доля ложноположительных:", false_positive_rate[116]
```

```
Порог: 0.5282247778873397
Доля истинноположительных: 0.810204081632653
Доля ложноположительных: 0.15490196078431373
```

Если же увеличить порог, например, до 0.8:

```
print("Порог:", threshold[45])
print("Доля истинноположительных:", true_positive_rate[45])
print("Доля ложноположительных:", false_positive_rate[45])
```

```
Порог: 0.8080195665631111
Доля истинноположительных: 0.563265306122449
Доля ложноположительных: 0.047058823529411764
```

Мы видим, что **при повышении порога TPR снижается**, так как модель становится осторожнее, но **FPR также уменьшается**, что делает модель более надёжной в контексте избегания ложных срабатываний.

Интерпретация ROC и AUC

ROC-кривая полезна не только как визуальный инструмент — она также позволяет вычислить **площадь под кривой (AUC — Area Under Curve)**.

Показатель **ROC-AUC** отражает общую способность модели различать классы при всех возможных порогах.

Если **AUC = 1**, модель идеально различает классы. Если **AUC = 0.5**, модель работает не лучше случайного угадывания.

```
# Вычислить площадь под кривой ROC
roc_auc_score(target_test, target_probabilities)
```

```
0.90733893557422962
```

ROC-кривая и метрика **AUC** — важные инструменты для оценки бинарных классификаторов, особенно в задачах с несбалансированными классами.

Они позволяют не просто измерить качество модели, но и понять, как изменяется её поведение при варьировании вероятностного порога, что может быть критично при выборе стратегии для конкретной бизнес-задачи.

1.3 Оценивание предсказаний мультиклассового классификатора

В отличие от бинарной классификации, где модель различает лишь два возможных исхода, **мультиклассовая классификация** предполагает работу с тремя и более классами. В таких задачах важно не только определить, правильно ли модель предсказывает конкретный класс, но и насколько равномерно она справляется с классификацией по всем категориям.

Оценка качества мультиклассового классификатора помогает понять, нет ли смещения модели в сторону отдельных классов и насколько уверенно она различает объекты разных типов. Для этого применяются

специальные метрики, которые обобщают принципы бинарной оценки на случай нескольких классов.

Одной из наиболее распространённых метрик является **accuracy** — доля верных предсказаний среди всех примеров. Однако при несбалансированных данных этого показателя может быть недостаточно, поэтому дополнительно используют такие метрики, как *средняя прецизионность*, *полнота*, *оценка* F_1 по каждому классу и их усреднённые варианты (*macro*, *micro* или *weighted*).

Пример кода на Python с использованием библиотеки **scikit-learn**:

```
# Загрузить библиотеки
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification

# Сгенерировать матрицу признаков и вектор целей
features, target = make_classification(
    n_samples=10000,
    n_features=3,
    n_informative=3,
    n_redundant=0,
    n_classes=3,
    random_state=1
)

# Создать объект логистической регрессии
logit = LogisticRegression()

# Перекрестно проверить модель, используя показатель точности
cross_val_score(logit, features, target, scoring='accuracy')
```

Результат:

```
array([0.83653269, 0.8259826 , 0.81308131])
```

Если классы сбалансированы — то есть содержат примерно одинаковое количество наблюдений, — метрика **точность (accuracy)** является

простой, интуитивно понятной и эффективной. Она показывает долю верно классифицированных объектов от общего числа.

Однако, если классы **несбалансированы** (что часто встречается в реальных данных), использование точности может быть вводящим в заблуждение. В таких случаях предпочтительно применять другие метрики, которые учитывают структуру распределения классов — например, **прецизионность (precision)**, **полноту (recall)** и **F1-меру (F1-score)**.

Библиотека **scikit-learn** предоставляет возможность вычислять эти метрики не только для бинарных, но и для мультиклассовых задач. Это достигается за счёт преобразования задачи в набор бинарных классификаций (один против всех, one-vs-rest), после чего метрики усредняются по классам с использованием различных стратегий:

- **macro** — вычислить среднее значение метрик по всем классам, придавая каждому классу одинаковый вес;
- **weighted** — усреднить метрики с учётом количества наблюдений в каждом классе (чем больше класс, тем больший вес);
- **micro** — рассчитать метрики на уровне всех комбинаций «класс-наблюдение», объединяя все классы в одну общую совокупность.

Пример расчёта **макроусреднённой F1-оценки**:

```
# Перекрестно проверить модель, используя макроусредненную оцен
ку F1
cross_val_score(logit, features, target, scoring='f1_macro')
```

Результат:

```
array([0.83613125, 0.82562258, 0.81293539])
```

Использование макроусреднения особенно полезно, когда важно оценить **равномерность качества предсказаний** модели по всем классам, а не просто общую точность. Если же классы сильно различаются по количеству наблюдений, взвешенное усреднение (**weighted**) даст более объективную оценку.

Таким образом, при оценке мультиклассовых классификаторов важно выбирать метрику, соответствующую особенностям данных и бизнес-

задачи. Например, при медицинской диагностике лучше минимизировать количество пропущенных положительных случаев (максимизировать полноту), а при фильтрации спама — уменьшить число ложных тревог (максимизировать прецизионность).

1.4 Визуализация результативности классификатора

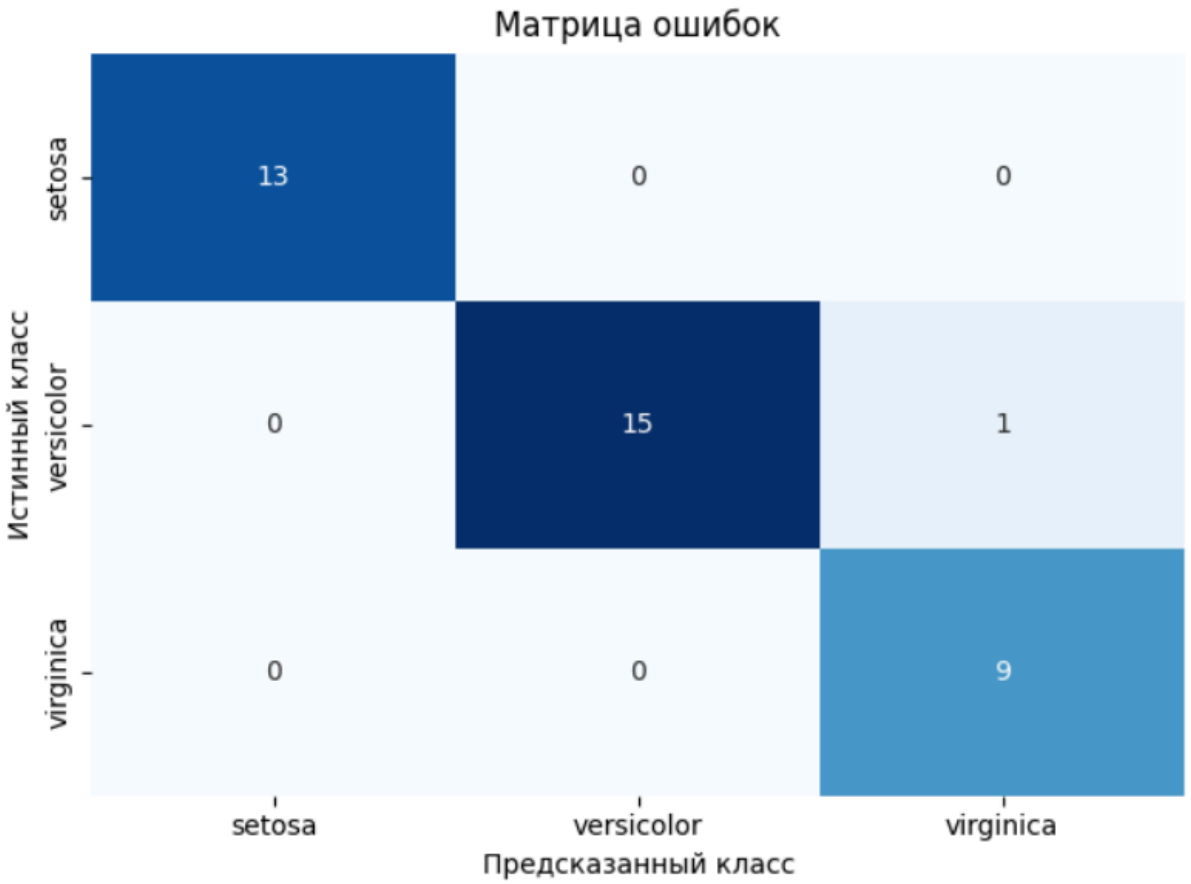
Визуализация позволяет не просто вычислить численные метрики качества классификации, но и наглядно увидеть, **как именно** модель принимает решения и где она чаще всего ошибается. Графическое представление результатов помогает быстро выявить закономерности в предсказаниях, определить, какие классы модель различает уверенно, а какие — путает между собой.

Одним из наиболее информативных инструментов визуального анализа является **матрица ошибок (confusion matrix)**. Она показывает распределение правильных и неправильных предсказаний между всеми классами, позволяя оценить типичные ошибки модели. Для удобства восприятия результаты часто представляют в виде **тепловой карты (heatmap)**, где интенсивность цвета отражает количество предсказаний в каждой категории.

Пример реализации на Python:

```
# Загрузить библиотеки
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import pandas as pd

# Загрузить данные
iris = datasets.load_iris()

# Создать матрицу признаков и вектор целей
features = iris.data
target = iris.target
```

```
# Получить имена классов
class_names = iris.target_names

# Разделить данные на обучающую и тестовую выборки
features_train, features_test, target_train, target_test = train_test_split(
    features, target, random_state=1
)

# Создать и обучить модель логистической регрессии
classifier = LogisticRegression(max_iter=200)
classifier.fit(features_train, target_train)

# Сделать предсказания
target_predicted = classifier.predict(features_test)

# Создать матрицу ошибок
matrix = confusion_matrix(target_test, target_predicted)

# Преобразовать в DataFrame для визуализации
dataframe = pd.DataFrame(matrix, index=class_names, columns=class_na
mes)

# Построить тепловую карту
sns.heatmap(dataframe, annot=True, cbar=None, cmap="Blues")
plt.title("Матрица ошибок")
plt.ylabel("Истинный класс")
plt.xlabel("Предсказанный класс")
plt.tight_layout()
plt.show()
```



Матрица ошибок — это один из самых информативных и простых способов оценить эффективность классификатора. Она демонстрирует, **насколько точно модель классифицирует объекты каждого класса**, и при этом позволяет понять **характер её ошибок**.

Каждая строка матрицы представляет **истинные классы**, а каждый столбец — **предсказанные классы**. Таким образом, элемент матрицы в позиции (i, j) показывает, сколько раз объект **истинного класса** i был классифицирован как **класс** j.

Например, в решении с набором данных *Iris*:

- Левая верхняя ячейка показывает количество экземпляров *Iris setosa*, которые модель правильно классифицировала как *Iris setosa*.
- Нижняя правая ячейка отображает количество верно предсказанных *Iris virginica*.
- Если посмотреть на ячейку выше — можно увидеть, что часть *Iris versicolor* была ошибочно отнесена к *Iris virginica*.

Такой анализ позволяет не только увидеть **количество ошибок**, но и понять **их природу**. Например, если модель часто путает *Iris versicolor* и *Iris virginica*, это может означать, что признаки этих классов сильно пересекаются в пространстве данных, и для улучшения модели стоит рассмотреть другие признаки или методы.

Важно отметить три ключевых момента при интерпретации матрицы ошибок:

1. **Идеальная модель** имеет значения только на диагонали матрицы (все предсказания верные) и нули в остальных ячейках.
2. **Плохая модель** имеет распределение ошибок, близкое к случайному — значения разбросаны по всей матрице.
3. **Анализ структуры ошибок** позволяет понять, в каких случаях модель ошибается чаще всего, и использовать это знание для улучшения алгоритма или данных (например, добавив новые признаки или увеличив количество примеров редких классов).

Матрица ошибок хорошо масштабируется для любого числа классов — от бинарных до многоклассовых задач. Однако при очень большом количестве классов её визуализация становится громоздкой, и тогда предпочтительно использовать агрегированные метрики, такие как **macro-F1** или **AUC ROC**, либо строить частичные матрицы по подмножествам классов.

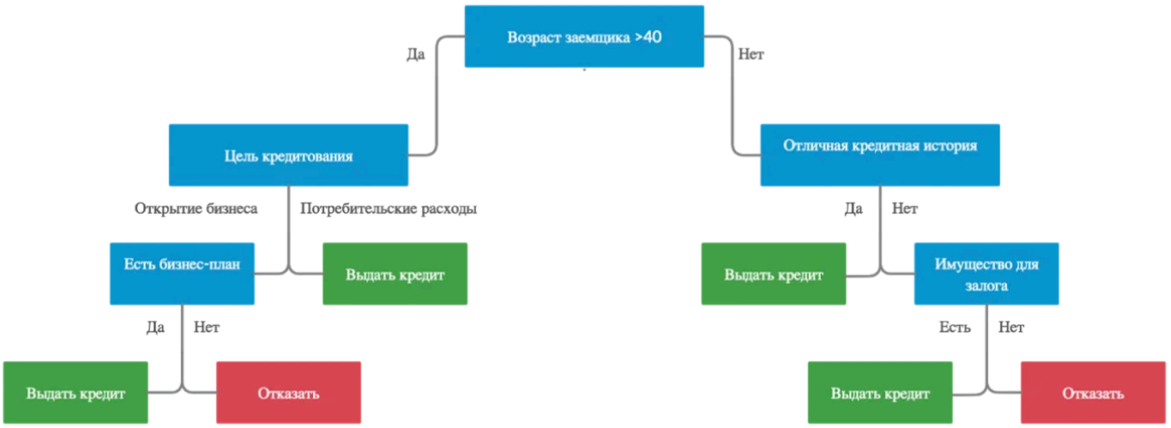
Более подробно почитать про метрики можно здесь:
https://education.yandex.ru/handbook/ml/article/metriki-klassifikacii-i-regressii

2 Деревья решений

Методы на основе деревьев решений — это одно из самых популярных и широко применяемых семейств **непараметрических методов машинного обучения с учителем**, используемых как для **классификации**, так и для **регрессии**. В основе этих алгоритмов лежит концепция **дерева решений** — структуры, в которой набор последовательных правил («если... то...») разбивает пространство признаков на подмножества, приводя к финальному прогнозу.

> Термин "непараметрический метод" означает, что модель не предполагает фиксированного числа параметров — структура дерева

> определяется данными и может быть произвольно сложной.



Дерево решений можно представить как **перевернутое дерево**:

- в верхней части (в корне) находится первое правило разделения;
- ниже располагаются **внутренние узлы**, где принимаются следующие решения;
- а **листья дерева** содержат итоговые предсказания модели.

Одной из причин популярности таких моделей является их **высокая интерпретируемость**. В отличие от нейронных сетей или ансамблей моделей, дерево решений можно буквально **нарисовать и объяснить**, показывая, какие признаки и на каких этапах влияют на результат.

От базового дерева решений произошли целые семейства более сложных моделей:

- **Случайные леса (Random Forests)** — ансамбли деревьев, уменьшающие переобучение;
- **Градиентный бустинг (Gradient Boosting Trees)** — метод, комбинирующий множество деревьев для последовательного улучшения предсказаний.

В этом разделе мы рассмотрим, как **тренировать, визуализировать и оценивать** различные модели на основе деревьев решений, а также разберем принципы их настройки.

2.1 Обучение дерева решений для задач классификации

В библиотеке `scikit-learn` реализация дерева решений для задач классификации представлена классом `DecisionTreeClassifier`, который позволяет гибко настраивать параметры модели — глубину дерева, критерий разбиения, минимальное число объектов в листьях и другие.

```
# Загрузить библиотеки
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets

# Загрузить данные
iris = datasets.load_iris()
features = iris.data
target = iris.target

# Создать объект-классификатор дерева принятия решений
decisiontree = DecisionTreeClassifier(random_state=0)

# Натренировать модель
model = decisiontree.fit(features, target)
```

Алгоритмы на основе дерева решений стремятся найти **правила разделения**, которые максимально уменьшают **разнородность данных в каждом узле**. Но как именно выбирается признак и порог для разделения? Интуитивно, "хорошее" разбиение должно создавать узлы, где преобладает один класс. Для количественной оценки этого свойства используются **меры чистоты узла**, среди которых наиболее распространены:

- **коэффициент Джини (Gini impurity)** — используется по умолчанию;
- **энтропия (Entropy)** — альтернативная мера, связанная с теорией информации.

Формула коэффициента Джини имеет вид:

$$G(t) = 1 - \sum_c p_c^2$$

где:

- $G(t)$ — коэффициент разнородности Джини в узле t;

- p_c— доля наблюдений класса **c** в этом узле.

Чем меньше значение $G(t)$, тем "чище" узел. Например:

- Если в узле все объекты одного класса: $G(t) = 1 - 1^2 = 0$ (идеальная чистота)
- Если в узле два класса распределены поровну: $G(t) = 1 - (0.5^2 + 0.5^2) = 0.5$ (максимальная неопределенность)

Алгоритм построения дерева (CART - Classification and Regression Trees) работает следующим образом:

1. Все данные находятся в корневом узле
2. **Для каждого узла:**
   - Перебираются признаки и пороги разбиения
   - Для каждого варианта разбиения вычисляется **прирост информации**:
     $\text{Information Gain} = G(\text{родитель}) - \left[\frac{N_{\text{левый}}}{N} \cdot G(\text{левый}) + \frac{N_{\text{правый}}}{N} \cdot G(\text{правый})\right]$
     где N — количество объектов в узлах
   - Выбирается разбиение с максимальным приростом информации
3. Процесс повторяется для каждого получившегося дочернего узла — сначала делится корневой узел, затем каждый получившийся узел делится дальше, и так до достижения критерия остановки
4. Алгоритм прекращает разбиение, когда:
   - Узел полностью однороден (все объекты одного класса)
   - Достигнута максимальная глубина дерева
   - Число объектов в узле меньше минимального порога
   - Прирост информации слишком мал

После обучения модель можно использовать для предсказаний:

```
# Сконструировать новое наблюдение
observation = [[5, 4, 3, 2]]

# Предсказать класс наблюдения
```

```
model.predict(observation)
# Вывод: array([1])
```

Модель не только сообщает, к какому классу отнесено наблюдение, но и позволяет оценить **вероятности принадлежности** к каждому классу:

```
# Предсказать вероятности для трех классов
model.predict_proba(observation)
# Вывод: array([[0., 1., 0.]])
```

Эти вероятности вычисляются как **доля объектов соответствующего класса в листе**, куда попало наблюдение. Например, если в конечном листе оказалось 30 объектов класса A и 10 объектов класса B, то вероятности будут [0.75, 0.25].

Это особенно полезно, когда необходимо учитывать **уверенность модели** при принятии решений — например, для установления вероятностных порогов или оценки рисков ошибочной классификации.

Если требуется изменить меру неоднородности, можно указать параметр `criterion` при создании модели:

```
# Создать классификатор дерева решений с использованием энтропии
decisiontree_entropy = DecisionTreeClassifier(
    criterion='entropy',
    random_state=0
)

# Натренировать модель
model_entropy = decisiontree_entropy.fit(features, target)
```

Энтропия определяется как:

$$H(t) = -\sum_{c} p_c \log_2(p_c)$$

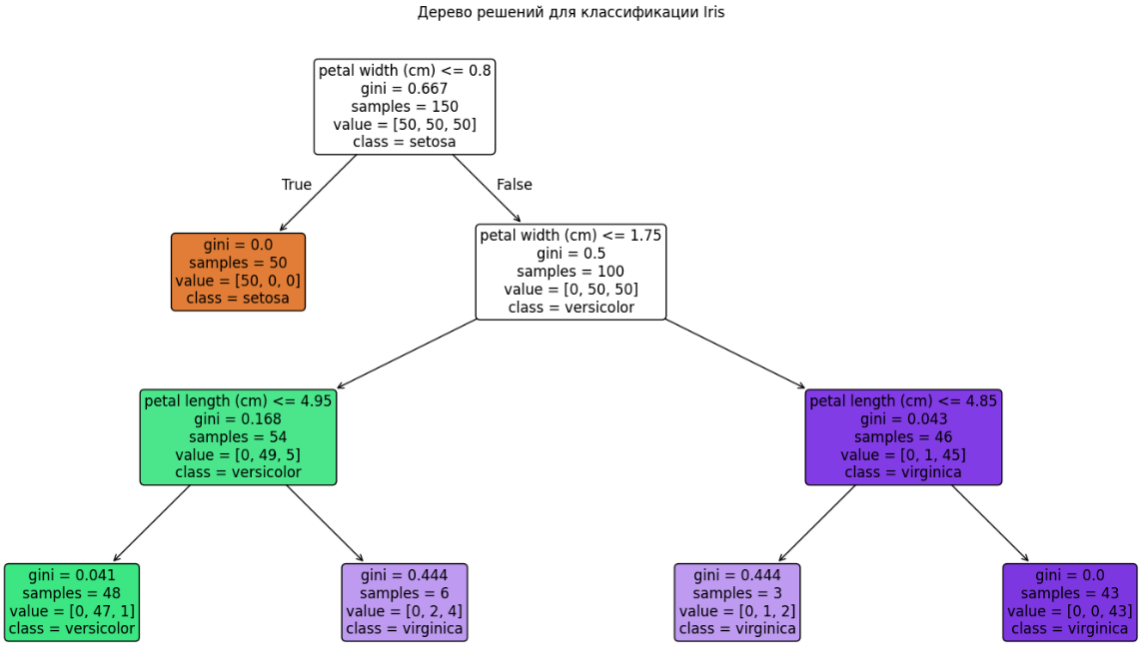
На практике выбор между Джини и энтропией редко критичен — результаты обычно очень похожи. Коэффициент Джини немного быстрее вычисляется, поэтому используется по умолчанию.

Визуализация дерева решений

Одно из главных преимуществ деревьев решений — возможность визуализировать процесс принятия решений. В **scikit-learn** для этого используется функция `plot_tree`:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

# Обучим дерево с ограниченной глубиной для наглядности
simple_tree = DecisionTreeClassifier(max_depth=3, random_state=0)
simple_tree.fit(features, target)

# Визуализация
plt.figure(figsize=(20, 10))
plot_tree(simple_tree,
          filled=True,  # Заливка узлов цветом по классу
          feature_names=iris.feature_names,
          class_names=iris.target_names,
          rounded=True,
          fontsize=12)
plt.title("Дерево решений для классификации Iris")
plt.show()
```

Результат:



Интерпретация визуализации:

- **Корневой узел** (вверху): показывает первое правило, например "petal length <= 2.45"
- **Цвет узла**: указывает на преобладающий класс (чем насыщеннее, тем однороднее узел)
- **samples**: количество объектов обучающей выборки в этом узле
- **value**: распределение объектов по классам [класс_0, класс_1, класс_2]
- **gini**: значение коэффициента Джини (чистота узла)

После обучения дерева можно оценить, какие признаки наиболее важны для классификации:

```
# Получить важность признаков
importances = model.feature_importances_

# Вывести с названиями признаков
for i, importance in enumerate(importances):
    print(f"{iris.feature_names[i]}: {importance:.3f}")
```

Пример вывода:

```
sepal length (cm): 0.000
sepal width (cm): 0.013
petal length (cm): 0.064
petal width (cm): 0.923
```

Важность признака показывает, насколько в среднем этот признак способствует уменьшению неопределенности (Джини или энтропии) при разбиениях. Сумма всех важностей равна 1.0.

В данном примере видно, что для классификации ирисов наиболее важны размеры лепестков (petal), а ширина чашелистика (sepal width) не используется.

2.2 Управление размером дерева

Контроль размера дерева решений — важнейший этап в построении устойчивой и обобщающей модели. Без ограничений дерево может расти слишком глубоко, подстраиваясь под отдельные особенности обучающей выборки и теряя способность правильно работать с новыми данными — это явление известно как **переобучение**.

Рассмотрим проблему на практическом примере:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Разделим данные на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.3, random_state=42
)

# Дерево без ограничений (может расти бесконечно)
deep_tree = DecisionTreeClassifier(random_state=0)
deep_tree.fit(X_train, y_train)

# Дерево с ограниченной глубиной
shallow_tree = DecisionTreeClassifier(max_depth=3, random_state=0)
shallow_tree.fit(X_train, y_train)
```

```
# Оценка на обучающей выборке
print("=== Точность на обучающей выборке ===")
print(f"Глубокое дерево: {accuracy_score(y_train, deep_tree.predict(X_train)):.3f}")
print(f"Ограниченное дерево: {accuracy_score(y_train, shallow_tree.predict(X_train)):.3f}")

# Оценка на тестовой выборке
print("\n=== Точность на тестовой выборке ===")
print(f"Глубокое дерево: {accuracy_score(y_test, deep_tree.predict(X_test)):.3f}")
print(f"Ограниченное дерево: {accuracy_score(y_test, shallow_tree.predict(X_test)):.3f}")
```

Типичный результат:

```
=== Точность на обучающей выборке ===
Глубокое дерево: 1.000
Ограниченное дерево: 0.971

=== Точность на тестовой выборке ===
Глубокое дерево: 0.956
Ограниченное дерево: 0.978
```

Анализ результатов:

- **Глубокое дерево** идеально классифицирует обучающие данные (100% точность), но на тестовых данных качество падает
- **Ограниченное дерево** показывает чуть меньшую точность на обучении, но лучше обобщает на новых данных

Параметры контроля структуры дерева

В библиотеке **scikit-learn** управление размером дерева осуществляется с помощью аргументов класса `DecisionTreeClassifier`. Грамотная настройка этих параметров помогает создать оптимальную модель, которая сохраняет высокое качество классификации при хорошей устойчивости к новым данным.

```
# Загрузить библиотеки
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets

# Загрузить данные
iris = datasets.load_iris()
features = iris.data
target = iris.target

# Создать объект-классификатор дерева решений с контролем структу
ры
decisiontree = DecisionTreeClassifier(
    random_state=0,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_leaf_nodes=None,
    min_impurity_decrease=0
)

# Обучить модель
model = decisiontree.fit(features, target)
```

Основные параметры регуляризации

Параметр	Описание	Эффект увеличения	Когда использовать
`max_depth`	Максимальная глубина дерева	Уменьшает переобучение	При переобучении (большой разрыв train/test)
`min_samples_split`	Мин. объектов для разбиения узла	Упрощает дерево	При переобучении, шумных данных
`min_samples_leaf`	Мин. объектов в листе	Создаёт более сглаженные границы	При переобучении, нужна стабильность
`max_leaf_nodes`	Макс. число листьев	Строго ограничивает	Для жесткого контроля размера

Параметр	Описание	Эффект увеличения	Когда использовать
		сложность	
`min_impurity_decrease`	Мин. прирост чистоты для разбиения	Предотвращает мелкие разбиения	При избыточной детализации

1. `max_depth` — максимальная глубина дерева

Если `None`, дерево растёт до тех пор, пока все листья не станут полностью однородными или не будут достигнуты другие критерии остановки.

Если указано целое число, то дерево "обрезается" до этой глубины.

```
# Пример: сравнение разных глубин
for depth in [2, 3, 5, None]:
    tree = DecisionTreeClassifier(max_depth=depth, random_state=0)
    tree.fit(X_train, y_train)
    train_acc = accuracy_score(y_train, tree.predict(X_train))
    test_acc = accuracy_score(y_test, tree.predict(X_test))
    print(f"max_depth={depth}: train={train_acc:.3f}, test={test_acc:.3f}")
```

Начинайте с `max_depth=3-5` для простых задач, `max_depth=10-15` для сложных. Если разрыв между train и test accuracy большой — уменьшайте глубину.

2. `min_samples_split` — минимальное количество наблюдений в узле для его разделения

Если указано целое число, задаётся абсолютный минимум.

Если передано вещественное число (например, `0.1`), оно означает процент от общего числа наблюдений.

```
# Пример: узлы делятся только при наличии не менее 20 наблюдений
tree = DecisionTreeClassifier(min_samples_split=20, random_state=0)
```

Значение по умолчанию (2) подходит для больших датасетов. Для малых выборок (< 1000 объектов) используйте `min_samples_split=10-50`.

3. `min_samples_leaf` — минимальное количество наблюдений в листе

Работает аналогично `min_samples_split`, но контролирует конечные узлы дерева (листья).

```
# Пример: каждый лист содержит не менее 10 объектов
tree = DecisionTreeClassifier(min_samples_leaf=10, random_state=0)
```

тот параметр часто эффективнее `min_samples_split` для борьбы с переобучением. Начните с `min_samples_leaf=5-20`.

4. `max_leaf_nodes` — максимальное количество листьев в дереве

Полезно для строгого ограничения числа конечных решений, что помогает уменьшить сложность модели.

```
# Пример: не более 10 конечных узлов (решений)
tree = DecisionTreeClassifier(max_leaf_nodes=10, random_state=0)
```

Используйте, когда нужна компактная модель с заранее заданным числом правил (например, для бизнес-логики).

5. `min_impurity_decrease` — минимальное требуемое уменьшение неопределенности

Узел будет разделён только в том случае, если прирост чистоты превышает этот порог.

```
# Пример: разделение только при существенном улучшении
tree = DecisionTreeClassifier(min_impurity_decrease=0.01, random_state=
0)
```

Начните с `0.001-0.01`. Полезно для предотвращения множества мелких незначительных разбиений.

Подбор гиперпараметров

Для систематического подбора оптимальных параметров используйте **решеточный поиск (Grid Search)**:

```
from sklearn.model_selection import GridSearchCV

# Определить сетку параметров для перебора
param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 5, 10],
```

```
    'criterion': ['gini', 'entropy']
}

# Создать объект GridSearch
grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=0),
    param_grid,
    cv=5,  # 5-fold кросс-валидация
    scoring='accuracy',
    n_jobs=-1  # использовать все ядра процессора
)

# Выполнить поиск
grid_search.fit(features, target)

# Результаты
print(f"Лучшие параметры: {grid_search.best_params_}")
print(f"Лучшая точность: {grid_search.best_score_:.3f}")
```

Для больших пространств параметров используйте `RandomizedSearchCV`, который тестирует случайное подмножество комбинаций.

2.3 Преимущества и недостатки дерева решений

Преимущества дерева решений

- **Простота интерпретации и визуализации** — модель можно представить в виде наглядной схемы, понятной даже непрофессионалу.
- **Отсутствие требований к масштабированию данных** — дерево корректно работает с признаками в любых единицах измерения.
- **Автоматическое выделение значимых признаков** — алгоритм самостоятельно определяет, какие переменные наиболее информативны.
- **Работа с признаками разного типа** — поддерживаются как числовые, так и категориальные данные.
- **Устойчивость к выбросам** — единичные аномалии в данных слабо влияют на структуру дерева.

- **Высокая скорость обучения и предсказания** — особенно на малых и средних объёмах данных.

Ограничения и недостатки дерева решений

- **Склонность к переобучению** — особенно без ограничений глубины дерева или при работе с шумными данными.
- **Нестабильность структуры** — небольшие изменения в обучающей выборке могут привести к совершенно другой модели.
- **Плохая способность к экстраполяции** — дерево не предсказывает значения за пределами диапазона обучающих данных.
- **Сложность моделирования сложных зависимостей** — для описания нелинейных или взаимодействующих признаков требуются очень глубокие деревья.
- **Ограниченная точность по сравнению с ансамблевыми методами** — одиночное дерево обычно уступает в качестве предсказаний случайному лесу или бустингу.

В целом, дерево решений — это мощный и наглядный инструмент для анализа данных, который служит фундаментом для более сложных ансамблевых моделей. Оно идеально подходит для первичного анализа, быстрого прототипирования и задач, где важна интерпретируемость решений.

3 K ближайших соседей

Метод *k ближайших соседей* (K-Nearest Neighbors, KNN) является одним из самых интуитивно понятных и широко используемых алгоритмов машинного обучения. Его основная идея заключается в том, что объекты, находящиеся близко друг к другу в пространстве признаков, с наибольшей вероятностью принадлежат одному и тому же классу.
В отличие от алгоритмов, которые в процессе обучения строят математическую модель (например, находят коэффициенты линейной регрессии), KNN относится к методам **ленивого обучения**: он просто сохраняет все обучающие данные в памяти и выполняет вычисления только в момент предсказания. Для определения класса нового наблюдения алгоритм ищет *k* ближайших точек из обучающей выборки и выбирает тот класс, который встречается среди них чаще всего.

Простота реализации и понятная логика работы делают KNN удобным инструментом как для обучения основам машинного обучения, так и для практического применения. Алгоритм может использоваться не только для **классификации**, но и для **регрессии**, когда значение предсказывается как среднее по соседним объектам.

3.1 Поиск ближайших соседей наблюдения

Для решения задачи поиска ближайших объектов в библиотеке `scikit-learn` применяется класс `NearestNeighbors`, который предоставляет инструменты для вычисления расстояний между наблюдениями и выбора *k* наиболее похожих точек.
В примере ниже мы загрузим известный набор данных **Iris**, стандартизируем его признаки с помощью `StandardScaler`, чтобы все переменные имели одинаковый масштаб, а затем с помощью `NearestNeighbors` найдём два ближайших соседа для нового наблюдения с заданными характеристиками. В завершение рассмотрим, какие объекты из обучающей выборки оказались самыми близкими к этому новому наблюдению.

```
# Загрузить библиотеки
from sklearn import datasets
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

# Загрузить данные
iris = datasets.load_iris()
features = iris.data

# Создать стандартизатор
standardizer = StandardScaler()

# Стандартизировать признаки
features_standardized = standardizer.fit_transform(features)

# Найти два ближайших соседа
nearest_neighbors = NearestNeighbors(n_neighbors=2).fit(features_standardized)
```

```
# Создать новое наблюдение
new_observation = [1, 1, 1, 1]

# Найти расстояния и индексы ближайших соседей
distances, indices = nearest_neighbors.kneighbors([new_observation])

# Посмотреть значения ближайших соседей
features_standardized[indices]
```

В данном примере каждый объект описывается четырьмя признаками цветка.

Мы создали фиктивное наблюдение `new_observation` и нашли **два ближайших** к нему наблюдения из обучающего набора.

- Переменная `indices` содержит индексы найденных соседей.
- Переменная `distances` — соответствующие расстояния до них.
- Значения `features_standardized[indices]` — сами соседние объекты.

Результат:

```
array([[[1.03800476, 0.56925129, 1.10395287, 1.1850097 ],
        [0.79566902, 0.33784833, 0.76275864, 1.05353673]]])
```

Метрики расстояния

В задачах, основанных на расстояниях, важно выбрать **подходящую метрику**.

В `scikit-learn` доступен широкий выбор метрик, среди которых:

- **Евклидово расстояние (Euclidean distance)**

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- **Манхэттенское расстояние (Manhattan distance)**

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

- **Расстояние Минковского (Minkowski distance)** — обобщённая форма обеих метрик:

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$$

  Здесь параметр *p* задаёт тип расстояния:

  - `p = 1` — Манхэттенское,
  - `p = 2` — Евклидово (используется по умолчанию).

Пример использования конкретной метрики:

```
nearestneighbors_euclidean = NearestNeighbors(
    n_neighbors=2, metric='euclidean'
).fit(features_standardized)

distances, indices = nearestneighbors_euclidean.kneighbors([new_observa
tion])
print(distances)
```

Результат:

```
array([[0.48168828, 0.73440155]])
```

Матрица ближайших соседей

Если нужно определить ближайших соседей **для каждого** объекта в выборке, можно воспользоваться методом `kneighbors_graph()`:

```
# Найти трех ближайших соседей каждого наблюдения (включая себя)
nearestneighbors_euclidean = NearestNeighbors(
    n_neighbors=3, metric="euclidean"
).fit(features_standardized)

# Создать матрицу соседей
nearest_neighbors_with_self = nearestneighbors_euclidean.kneighbors_gra
ph(
    features_standardized
```

```
).toarray()

# Удалить единицы (наблюдение не считается соседом самого себя)
for i, x in enumerate(nearest_neighbors_with_self):
    x[i] = 0

# Посмотреть соседей первого наблюдения
nearest_neighbors_with_self[0]
```

Когда алгоритм опирается на метрики расстояния, **масштаб признаков критически важен**.

Если один признак измеряется в тысячах, а другой в долях, то первый будет доминировать в расчёте расстояния.

Поэтому перед использованием KNN **обязательно нужно стандартизировать или нормализовать** признаки.

Мы сделали это с помощью `StandardScaler`, который приводит каждый признак к нулевому среднему и единичному стандартному отклонению.

Альтернативно можно использовать `MinMaxScaler`, если значения должны быть приведены к диапазону `[0, 1]`.

3.2 Создание классификационной модели k ближайших соседей

В этом разделе рассмотрим использование класса `KNeighborsClassifier` из библиотеки `scikit-learn` для построения модели на примере классического набора данных **Iris**. Мы обучим классификатор, выполним предсказание класса для нового наблюдения и проанализируем результаты работы модели.

```
# Загрузить библиотеки
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn import datasets

# Загрузить данные
iris = datasets.load_iris()
X = iris.data
```

```
y = iris.target

# Создать стандартизатор
standardizer = StandardScaler()

# Стандартизировать признаки
X_std = standardizer.fit_transform(X)

# Натренировать классификатор KNN с 5 соседями
knn = KNeighborsClassifier(n_neighbors=5, n_jobs=-1).fit(X_std, y)

# Создать два наблюдения
new_observations = [[0.75, 0.75, 0.75, 0.75],
                    [1, 1, 1, 1]]

# Предсказать класс двух наблюдений
knn.predict(new_observations)
```

Результат:

```
array([1, 2])
```

Алгоритм **k ближайших соседей** работает просто и интуитивно понятно.

Если имеется наблюдение x_i с неизвестным целевым классом, алгоритм сначала находит **k ближайших наблюдений** (так называемую **окрестность** точки x_i) на основе выбранной **метрики расстояния** (например, евклидова расстояния).

После этого эти k наблюдений «голосуют» за свой класс. Класс, который получает **наибольшее количество голосов**, становится **предсказанным классом** для данного наблюдения.

Более формально вероятность принадлежности точки x_i к классу **j** можно выразить так:

$$P(y = j|x_i) = \frac{1}{k} \sum_{v \in N_k(x_i)} I(y_v = j)$$

где

- $N_k(xi)$ — множество k ближайших соседей точки x_i,
- I— индикаторная функция (равна 1, если условие истинно, иначе 0).

В библиотеке `scikit-learn` эти вероятности можно получить с помощью метода `predict_proba()`:

```
# Посмотреть вероятность принадлежности каждого наблюдения к кла
ссам
knn.predict_proba(new_observations)
```

Результат:

```
array([[0. , 0.6, 0.4],
       [0. , 0. , 1. ]])
```

Первое наблюдение с вероятностью 0.6 принадлежит к классу 1, а второе — с вероятностью 1.0 к классу 2.

Метод `predict()` возвращает именно тот класс, который имеет **наибольшую вероятность**:

```
knn.predict(new_observations)
array([1, 2])
```

Важные параметры KNeighborsClassifier

1. `metric` — задает используемую метрику расстояния. По умолчанию это расстояние Минковского (с $p = 2$), что эквивалентно **евклидову расстоянию**. При желании можно использовать, например, `metric='manhattan'`.
2. `n_jobs` — определяет количество процессорных ядер, которые будут использоваться для вычислений.

   Так как алгоритм KNN требует вычисления расстояний от каждой точки до всех остальных, использование нескольких ядер **значительно ускоряет** процесс.
3. `algorithm` — задает метод поиска ближайших соседей (`'auto'`, `'ball_tree'`, `'kd_tree'`, `'brute'`).

Обычно можно оставить `'auto'`, и `scikit-learn` сам выберет оптимальный вариант в зависимости от данных.

4. `weights` — задает, каким образом учитывать соседей при голосовании.

   По умолчанию все соседи имеют **равные веса** (`weights='uniform'`), однако можно установить `weights='distance'`, чтобы **более близкие соседи имели больший вес** при голосовании.

   Это особенно полезно, когда плотность данных неравномерна.

3.3 Идентификация наилучшего размера окрестности

Одним из ключевых параметров модели *k* ближайших соседей является число соседей — **k**, от которого напрямую зависит точность классификации. Если значение *k* слишком мало, модель становится чувствительной к шуму и может переобучиться. Если же *k* слишком велико, классификатор теряет детализацию и сглаживает различия между классами.

Поэтому важно подобрать такое значение *k*, при котором модель демонстрирует наилучший баланс между точностью и устойчивостью. Для этого применяется процедура автоматического подбора гиперпараметров — **поиск по сетке параметров**, реализованный в библиотеке `scikit-learn` с помощью класса `GridSearchCV`.

```
# Загрузить библиотеки
from sklearn.neighbors import KNeighborsClassifier
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

# Загрузить данные
iris = datasets.load_iris()
features = iris.data
target = iris.target

# Создать стандартизатор
standardizer = StandardScaler()
```

```
# Стандартизировать признаки
features_standardized = standardizer.fit_transform(features)

# Создать классификатор KNN
knn = KNeighborsClassifier(n_neighbors=5, n_jobs=-1)

# Создать конвейер (pipeline)
pipe = Pipeline([
    ("standardizer", standardizer),
    ("knn", knn)
])

# Задать пространство значений для k
search_space = [{"knn__n_neighbors": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]}]

# Создать и выполнить решеточный поиск
classifier = GridSearchCV(
    pipe,
    search_space,
    cv=5,        # 5-блочная кросс-валидация
    verbose=0
).fit(features_standardized, target)
```

Метод `GridSearchCV` выполняет поиск по заранее заданному диапазону значений k и использует **кросс-валидацию** для оценки качества каждой модели. Кросс-валидация делит выборку на несколько блоков (в нашем случае — 5) и последовательно обучает модель на одной части, проверяя на другой. Это позволяет избежать случайных перекосов и получить более надежную оценку качества модели.

После выполнения поиска можно получить оптимальное значение k:

```
# Наилучший размер окрестности (значение k)
classifier.best_estimator_.get_params()["knn__n_neighbors"]
```

Например, если результат равен `k = 6`, это значит, что использование 6 ближайших соседей даёт **наилучшее качество классификации** для данного набора данных.

3.3 Преимущества и недостатки k ближайших соседей

Преимущества метода k ближайших соседей:

- Простота реализации — алгоритм легко понять и применить даже при минимальных знаниях машинного обучения.
- Отсутствие этапа обучения — метод не требует построения модели; все вычисления происходят на этапе предсказания.
- Естественная адаптация к сложным распределениям — метод хорошо работает в случаях, когда границы классов имеют сложную, нелинейную форму.

Ограничения и недостатки метода k ближайших соседей:

- Высокие вычислительные затраты при предсказании — для каждого нового объекта требуется рассчитать расстояние до всех точек обучающей выборки, что делает метод неэффективным на больших объёмах данных.
- Чувствительность к масштабированию признаков — алгоритм напрямую зависит от расстояний, поэтому данные требуют нормализации или стандартизации.
- Влияние шума и нерелевантных признаков — наличие выбросов или неинформативных признаков может существенно исказить результаты.
- Неэффективность при большом числе признаков — в условиях "проклятия размерности" расстояния между точками становятся менее информативными, что снижает точность метода.

В целом, метод k ближайших соседей — это интуитивно понятный и надёжный базовый алгоритм, который демонстрирует хорошие результаты на небольших и хорошо структурированных данных.

4 Логистическая регрессия

Логистическая регрессия — один из базовых и в то же время мощных методов **классификации** в машинном обучении. Несмотря на название, этот алгоритм не решает задачи регрессии в привычном смысле, а используется для **прогнозирования вероятности принадлежности объекта к определённому классу**.

Метод основан на применении **логистической функции (сигмоиды)**, которая преобразует линейную комбинацию признаков в значение вероятности от 0 до 1. Это делает логистическую регрессию особенно удобной для решения **бинарных задач классификации**, а её обобщения — для **многоклассовых сценариев**.

4.1 Логистическая регрессия для бинарной классификации

В этом разделе мы рассмотрим, как с помощью библиотеки **scikit-learn** обучить модель логистической регрессии с помощью класса `LogisticRegression`, проанализировать коэффициенты модели и использовать её для предсказания класса новых наблюдений на примере данных Iris. Перед обучением модели необходимо **стандартизировать признаки**. Логистическая регрессия чувствительна к масштабам данных: признаки с большими значениями могут доминировать в процессе обучения, что приведёт к некорректным результатам. Стандартизация приводит все признаки к одному масштабу, обеспечивая равноценный вклад каждого признака в модель.

```
# Загрузить библиотеки
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Загрузить данные только с двумя классами
iris = datasets.load_iris()
features = iris.data[:100, :]
target = iris.target[:100]

# Разделить данные на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.2, random_state=0
)

# Стандартизировать признаки
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
```

```
X_test_std = scaler.transform(X_test)

# Создать объект логистической регрессии
logistic_regression = LogisticRegression(random_state=0)

# Обучить модель на обучающей выборке
model = logistic_regression.fit(X_train_std, y_train)
```

Несмотря на название, **логистическая регрессия** не является моделью регрессии в классическом смысле. На практике это один из самых распространённых **бинарных классификаторов**, используемых для задач, где целевая переменная принимает одно из двух возможных значений (например, *да/нет*, *0/1*, *положительный/отрицательный* и т.д.).

Математически модель логистической регрессии основывается на линейной комбинации входных признаков, которая затем преобразуется с помощью **логистической функции (сигмоиды)**.

Формула модели имеет вид:

$$P(yi = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}}$$

где:

- $P(y_i = 1 \mid X)$— вероятность, что объект i принадлежит к классу 1,
- $\beta_0, \beta_1, \ldots, \beta_n$ — параметры модели,
- $X = (x_1, x_2, \ldots, x_n)$— вектор признаков наблюдения,
- e — основание натурального логарифма.

Сигмоидальная функция ограничивает выход модели диапазоном **от 0 до 1**, что делает возможной интерпретацию результата как вероятности.

Если значение вероятности больше 0.5, модель предсказывает класс **1**; иначе — класс **0**.

После обучения модели можно использовать её для классификации новых наблюдений:

```
# Создать новое наблюдение (необходимо стандартизировать)
new_observation = [[0.5, 0.5, 0.5, 0.5]]
```

```
new_observation_std = scaler.transform(new_observation)

# Предсказать класс
predicted_class = model.predict(new_observation_std)
print("Предсказанный класс:", predicted_class)
# Вывод:
# Предсказанный класс: [0]
```

В данном примере новое наблюдение классифицируется как класс **1**.

Чтобы узнать не только предсказанный класс, но и **вероятность принадлежности** к каждому из классов, используется метод `predict_proba`:

```
# Получить предсказанные вероятности
predicted_proba = model.predict_proba(new_observation_std)
print("Вероятности принадлежности к классам:", predicted_proba)
# Вывод:
# Вероятности принадлежности к классам: [[0.96587518 0.03412482]]
```

Это означает, что вероятность того, что наблюдение принадлежит классу 0, составляет 96**.6%**, а вероятность того, что оно принадлежит классу 1, — 3.**4%**.

Одним из преимуществ логистической регрессии является **интерпретируемость коэффициентов модели**. Каждый коэффициент показывает, как изменение конкретного признака влияет на вероятность отнесения наблюдения к классу.

```
# Получить коэффициенты модели
coefficients = model.coef_[0]
intercept = model.intercept_[0]

print("Коэффициенты признаков:", coefficients)
print("Свободный член (intercept):", intercept)

# Визуализация важности признаков
feature_names = ['sepal length', 'sepal width', 'petal length', 'petal width']
import pandas as pd

feature_importance = pd.DataFrame({
```

```
    'Признак': feature_names,
    'Коэффициент': coefficients
}).sort_values('Коэффициент', key=abs, ascending=False)

print("\nВажность признаков:")
print(feature_importance)
```

Результат:

```
Коэффициенты признаков: [ 0.77990721 -1.09825262  1.46929017  1.4757
3277]
Свободный член (intercept): 0.17249905643885408

Важность признаков:
        Признак  Коэффициент
3   petal width     1.475733
2  petal length     1.469290
1   sepal width    -1.098253
0  sepal length     0.779907
```

Интерпретация коэффициентов:

- **Положительный коэффициент**: увеличение значения признака повышает вероятность отнесения к классу 1
- **Отрицательный коэффициент**: увеличение значения признака снижает вероятность отнесения к классу 1
- **Модуль коэффициента**: показывает силу влияния признака — чем больше модуль, тем сильнее влияние

Например, если коэффициент для длины лепестка равен 2.5, это означает, что при увеличении длины лепестка на одну единицу (после стандартизации) логит вероятности класса 1 увеличивается на 2.5.

4.2 Обучение мультиклассового классификатора

Логистическая регрессия изначально является **бинарным классификатором**, то есть способна различать только два класса. Однако на практике часто требуется классифицировать данные на три и более

категории — например, определять вид растения по его морфологическим признакам, как в наборе данных **Iris**.

Для решения таких задач в scikit-learn предусмотрены два подхода:

1. **Метод "один против остальных" (One-vs-Rest, OvR)**

   В этом подходе для каждого класса создается отдельная бинарная модель. Каждая модель учится различать, относится ли наблюдение к данному классу или нет.

   Например, если у нас есть три класса — 0, 1 и 2 — то будет построено три модели:

   - модель 1: класс 0 против (1 и 2);
   - модель 2: класс 1 против (0 и 2);
   - модель 3: класс 2 против (0 и 1).

   При предсказании каждая модель возвращает вероятность, и объект относят к тому классу, для которого вероятность наибольшая.

   Этот метод прост, интерпретируем и подходит для большинства задач, особенно когда классы не сильно пересекаются.

2. **Мультиклассовая (полиномиальная) логистическая регрессия (Multinomial)**

   В отличие от OvR, этот подход не разбивает задачу на несколько бинарных подзадач. Он напрямую оценивает вероятность принадлежности к каждому из классов одновременно, используя обобщенную форму логистической функции.

   Этот метод особенно полезен, когда данные хорошо сбалансированы и классы не независимы друг от друга.

   Пример реализации метода OvR в Python:

```
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.multiclass import OneVsRestClassifier

# Загрузить данные
```

```
iris = datasets.load_iris()
features = iris.data
target = iris.target

# Разделить на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.2, random_state=0, stratify=target
)

# Стандартизировать признаки
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Обернуть логистическую регрессию в OneVsRestClassifier
logistic_regression = OneVsRestClassifier(LogisticRegression(random_state
=0))

# Обучить модель
model = logistic_regression.fit(X_train_std, y_train)

# Оценить точность на тестовой выборке
accuracy = model.score(X_test_std, y_test)
print(f"Точность модели OvR: {accuracy:.2f}")
```

Логистическая регрессия **всегда** использует `"multinomial"`, если это поддерживает выбранный `solver` (например, `lbfgs`, `saga`, `newton-cg`):

```
logistic_regression_multi = LogisticRegression(
    random_state=0,
    solver="lbfgs"  # поддерживает multinomial автоматически
)

# Обучить модель
model_multi = logistic_regression_multi.fit(X_train_std, y_train)

# Оценить точность
```

```
accuracy_multi = model_multi.score(X_test_std, y_test)
print(f"Точность модели Multinomial: {accuracy_multi:.2f}")
```

После обучения можно использовать модель для предсказания классов и вероятностей:

```
# Пример предсказания
new_observation = [[5.1, 3.5, 1.4, 0.2]]
predicted_class = model.predict(new_observation)
predicted_prob = model.predict_proba(new_observation)

print("Предсказанный класс:", predicted_class)
print("Вероятности принадлежности к классам:", predicted_prob)
```

Результат выполнения:

```
Предсказанный класс: [2]
Вероятности принадлежности к классам: [[0.00412374 0.01019325 0.98
568301]]
```

Таким образом, логистическая регрессия, будучи простым и интерпретируемым методом, позволяет эффективно решать как **бинарные**, так и **мультиклассовые** задачи классификации.

Выбор между OvR и Multinomial

- **OvR** лучше подходит для несбалансированных данных и когда классы независимы
- **Multinomial** эффективнее для сбалансированных данных и взаимосвязанных классов
- На практике рекомендуется попробовать оба подхода и выбрать по результатам кросс-валидации

4.3 Снижение дисперсии с помощью регуляризации

При обучении логистической регрессии модель может стать слишком сложной и начать «запоминать» обучающие данные, вместо того чтобы выявлять общие закономерности. Чтобы сделать модель более устойчивой

и улучшить её способность обобщать, применяется метод **регуляризации**, который ограничивает величину весов признаков, предотвращая их чрезмерное влияние. В библиотеке **scikit-learn** это достигается с помощью класса `LogisticRegressionCV`, позволяющего автоматически подобрать оптимальный уровень регуляризации (`C`) на основе кросс-валидации.

```
# Загрузить библиотеки
from sklearn.linear_model import LogisticRegressionCV
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Загрузить данные
iris = datasets.load_iris()
features = iris.data
target = iris.target

# Разделить на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.2, random_state=0, stratify=target
)

# Стандартизировать признаки
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Создать объект логистической регрессии с кросс-валидацией
logistic_regression = LogisticRegressionCV(
    penalty='l2',
    Cs=10,
    cv=5,  # количество fold для кросс-валидации
    random_state=0,
    n_jobs=-1,
    max_iter=1000
)

# Обучить модель
```

```
model = logistic_regression.fit(X_train_std, y_train)

# Оценить качество
accuracy = model.score(X_test_std, y_test)
print(f"Точность модели: {accuracy:.2f}")
```

Регуляризация — это метод, который **ограничивает сложность модели**, добавляя **штрафной член** к функции потерь. Ее основная цель — предотвратить **переобучение** и снизить **дисперсию модели**, сохранив при этом достаточную гибкость для обобщения данных.

В логистической регрессии обычно используются два типа регуляризации:

1. **L1-регуляризация (Lasso)** — добавляет штраф, пропорциональный сумме модулей весов.

   Она способствует **разреживанию модели**, зануляя некоторые коэффициенты, что упрощает модель и может использоваться для **отбора признаков**.

   Формула штрафа:

$$\lambda \sum_{i=1}^{n} |w_i|$$

2. **L2-регуляризация (Ridge)** — добавляет штраф, пропорциональный сумме квадратов весов.

   Она **не зануляет веса**, но уменьшает их значения, снижая влияние признаков с низкой информативностью и повышая устойчивость модели.

   Формула штрафа:

$$\lambda \sum_{i=1}^{n} w_i^2$$

Параметр **λ (лямбда)** управляет силой регуляризации — чем больше λ, тем сильнее штраф, и тем проще модель.

Однако в библиотеке **scikit-learn** принято использовать не λ, а **его обратную величину — C**

$$C = \frac{1}{\lambda}$$

Таким образом:

- малое значение **C** → сильная регуляризация (простая модель, меньше переобучения);
- большое значение **C** → слабая регуляризация (сложная модель, выше риск переобучения).

Класс `LogisticRegressionCV` выполняет **кросс-валидацию**, чтобы автоматически подобрать наилучшее значение **C**.

Параметр `Cs` определяет диапазон или количество тестируемых значений — если передано целое число, scikit-learn создаст логарифмическую шкалу от $1e-4$ **до** $1e4$.

После обучения можно посмотреть, какое значение регуляризации оказалось оптимальным:

```
# Оптимальное значение C
print("Лучшее значение C:", model.C_)
```

Пример вывода:

```
Лучшее значение C: [3.59381366]
```

Таким образом, регуляризация помогает **упростить модель, уменьшить дисперсию и повысить способность к обобщению**, особенно при работе с небольшими или зашумленными наборами данных.

4.4 Обработка несбалансированных классов

Несбалансированные классы — одна из самых распространённых проблем в машинном обучении, особенно при решении задач, где **редкий класс** имеет критическое значение.

Примеры реальных задач:

- **Обнаружение мошенничества**: мошеннические транзакции составляют < 1% от всех операций

- **Медицинская диагностика**: редкие заболевания встречаются у небольшого процента пациентов
- **Промышленность**: обнаружение дефектов или отказов оборудования
- **Кредитный скоринг**: дефолты по кредитам составляют малую долю

Если модель обучается на несбалансированных данных без корректировок, она может просто «игнорировать» редкий класс, потому что минимизация ошибки в этом случае проще достигается за счёт правильного предсказания большинства.

Например, если только 1% транзакций мошеннические, модель может достичь 99% точности, просто всегда предсказывая «не мошенничество», что бесполезно на практике.

В **scikit-learn** предусмотрен встроенный механизм компенсации этой проблемы с помощью параметра `class_weight`.

```
# Загрузить библиотеки
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.preprocessing import StandardScaler

# Загрузить данные
iris = datasets.load_iris()
features = iris.data
target = iris.target

# Сделать классы несбалансированными, удалив первые 40 наблюден
ий
features = features[40:, :]
target = target[40:]

# Создать вектор целей: только два класса (0 и 1)
target = np.where(target == 0, 0, 1)

# Стандартизировать признаки
scaler = StandardScaler()
features_standardized = scaler.fit_transform(features)
```

```
# Создать логистическую регрессию с автоматическим балансом клас
сов
logistic_regression = LogisticRegression(random_state=0, class_weight="b
alanced")

# Натренировать модель
model = logistic_regression.fit(features_standardized, target)
```

Если установить `class_weight="balanced"`, библиотека автоматически вычислит веса классов, **обратно пропорциональные их частоте в выборке**, по следующей формуле:

$$w_j = \frac{n}{k \cdot n_j}$$

где:

- w_j — вес для класса *j*;
- n — общее количество наблюдений;
- n_j — количество наблюдений в классе *j*;
- k — общее количество классов.

Таким образом, **редкие классы получают больший вес**, а частые — меньший. Это позволяет уравновесить вклад каждого класса в функцию потерь и повысить точность модели для миноритарных (редких) классов.

Кроме автоматического баланса `balanced`, параметр `class_weight` можно задать вручную в виде словаря, например:

```
# Пример ручного задания весов
weights = {0: 0.3, 1: 0.7}
logistic_regression = LogisticRegression(random_state=0, class_weight=w
eights)
```

Такой подход бывает полезен, когда известно, что ошибки для некоторых классов **более критичны**, чем для других.

Важно помнить:

Регуляризация `C`, стандартизация признаков и балансировка классов — это три взаимосвязанных шага, которые в совокупности позволяют добиться устойчивой и адекватной модели логистической регрессии даже на сложных, реальных данных.

4.5 Преимущества и недостатки логистической регрессии

Преимущества логистической регрессии:

- Простота интерпретации — результаты модели легко объяснить: коэффициенты показывают влияние каждого признака на вероятность исхода.
- Высокая эффективность при линейно разделимых данных — логистическая регрессия хорошо работает, если классы можно разделить гиперплоскостью.
- Малое количество вычислительных ресурсов — обучение модели занимает немного времени и не требует больших объёмов памяти.
- Возможность оценки вероятностей — в отличие от многих других методов классификации, логистическая регрессия предоставляет не только метку класса, но и вероятность принадлежности к нему.
- Устойчивость к переобучению при правильной регуляризации — применение L1 или L2-регуляризации помогает контролировать сложность модели и снижает риск переобучения.

Ограничения и недостатки логистической регрессии:

- Ограниченная способность моделировать нелинейные зависимости — базовая логистическая регрессия предполагает линейную связь между признаками и логистической функцией, что снижает точность на сложных данных.
- Чувствительность к мультиколлинеарности — сильная корреляция между признаками может исказить оценки коэффициентов и ухудшить интерпретацию.
- Требовательность к масштабированию данных — при различиях в масштабе признаков могут возникнуть проблемы с сходимостью алгоритма обучения.

- Неэффективность при большом количестве неинформативных признаков — модель теряет точность, если данные содержат шум или лишние переменные.
- Ограниченная применимость для задач с сильно несбалансированными классами — при диспропорции классов требуется дополнительная настройка весов или использование специализированных методов.

5 Наивный байесовский классификатор

Теорема Байеса является основополагающим инструментом для понимания вероятности наступления некоторого события $P(A \mid B)$ при наличии новой информации — вероятности $P(B \mid A)$ и априорной оценки вероятности события $P(A)$.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

В области машинного обучения применение теоремы Байеса наиболее наглядно проявляется в **наивном байесовском классификаторе**, который является одной из ключевых моделей для решения задач классификации.

Наивные байесовские классификаторы обладают рядом преимуществ, делающих их удобными и эффективными для практического использования:

- интуитивно понятный подход;
- способность работать даже с небольшими выборками данных;
- низкие вычислительные затраты при обучении и предсказании;
- стабильные и часто надёжные результаты в различных условиях.

Основная формула наивного Байеса

Наивный байесовский классификатор основан на применении теоремы Байеса для оценки вероятности принадлежности наблюдения к определённому классу. Общая формула записывается следующим образом:

$$P(y \mid x_1, ..., x_j) = \frac{P(x_1, ..., x_j \mid y) \cdot P(y)}{P(y|x_1, ..., x_j)}$$

где:

- $P(y \mid x_1, ..., x_j)$— **апостериорная вероятность**, то есть вероятность того, что наблюдение принадлежит классу y при условии, что его признаки равны $x_1, ..., x_j$;
- $P(x_1, ..., x_j \mid y)$ — **правдоподобие**, то есть вероятность наблюдать такие значения признаков при условии, что известен класс y;
- $P(y)$ — **априорная вероятность** класса y, представляющая субъективную оценку вероятности появления этого класса до анализа данных.
- $P(x_1, \ldots, x_j)$ называется **предельной вероятностью**.

В процессе классификации мы сравниваем апостериорные вероятности для всех возможных классов. Так как знаменатель $P(x_1, ..., x_j)$ одинаков для всех, достаточно сравнивать только числители — произведения $P(x_1, ..., x_j|y) \cdot P(y)$. Класс, для которого это произведение максимально, и становится предсказанным.

Название «наивный Байес» связано с главным допущением модели — **независимостью признаков** друг от друга. То есть, предполагается, что каждый признак вносит вклад в классификацию независимо от остальных. На практике это допущение часто нарушается, но, несмотря на «наивность», модель остаётся эффективной и часто показывает отличные результаты даже при наличии зависимостей между признаками.

5.1 Тренировка классификатора для непрерывных признаков

Рассмотрим случай, когда все признаки в наборе данных являются **непрерывными**, и требуется обучить наивный байесовский классификатор. В этом случае используется **гауссов наивный Байес** (Gaussian Naive Bayes), который предполагает, что значения признаков подчиняются нормальному (гауссовому) распределению.

Для реализации гауссова наивного байесовского классификатора в можно воспользоваться классом `GaussianNB` из библиотеки **scikit-learn**. Ниже

приведён пример базовой настройки и обучения модели на классическом наборе данных **Iris**:

```
# Загрузить необходимые библиотеки
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, accuracy_score, confusio
n_matrix

# Загрузить данные Iris (150 образцов, 4 признака, 3 класса)
iris = datasets.load_iris()
features = iris.data  # 4 признака: длина/ширина чашелистика и лепестк
a
target = iris.target  # Классы: 0 (setosa), 1 (versicolor), 2 (virginica)

# Разделить данные на обучающую (70%) и тестовую (30%) выборки
X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.3, random_state=42, stratify=target
)

# Создать объект гауссова наивного Байеса
classifier = GaussianNB()

# Обучить модель на тренировочных данных
model = classifier.fit(X_train, y_train)

# Предсказать классы для тестовой выборки
y_pred = model.predict(X_test)

# Оценить качество модели
print("Точность модели:", accuracy_score(y_test, y_pred))
print("\nОтчет по классификации:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
print("\nМатрица ошибок:")
print(confusion_matrix(y_test, y_pred))
```

Результат:

```
Точность модели: 0.9111111111111111

Отчет по классификации:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.82      0.93      0.88        15
   virginica       0.92      0.80      0.86        15

    accuracy                           0.91        45
   macro avg       0.92      0.91      0.91        45
weighted avg       0.92      0.91      0.91        45


Матрица ошибок:
[[15  0  0]
 [ 0 14  1]
 [ 0  3 12]]
```

Гауссов наивный Байес является самым распространённым вариантом наивных байесовских классификаторов. Его ключевое предположение заключается в том, что **значения признаков внутри каждого класса распределены по нормальному закону**.

Для каждого признака X_j, принадлежащего классу **y**, вычисляются параметры нормального распределения — **среднее значение** μ_y и **дисперсия** σ_y^2. Вероятность наблюдения значения признака при условии данного класса описывается функцией плотности нормального распределения:

$$P(xj \mid y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \cdot e^{-\frac{(x_j-\mu_y)^2}{2\sigma_y^2}}$$

Благодаря этому допущению гауссов наивный Байес особенно хорошо подходит для задач, где все признаки непрерывные (например, измерения, оценки, физические параметры).

Пример предсказания

После обучения модели можно использовать её для предсказания класса нового наблюдения. Например:

```
# Создать новое наблюдение (длина чашелистика=4.0, ширина=4.0,
# длина лепестка=4.0, ширина лепестка=0.4)
new_observation = [[4.0, 4.0, 4.0, 0.4]]

# Предсказать класс
predicted_class = model.predict(new_observation)
print(f"Предсказанный класс: {predicted_class[0]} ({iris.target_names[pre
dicted_class[0]]})")

# Получить вероятности для всех классов
probabilities = model.predict_proba(new_observation)
print(f"\nВероятности классов:")
for i, prob in enumerate(probabilities[0]):
    print(f"  {iris.target_names[i]}: {prob:.4f} ({prob*100:.2f}%)")
```

Результат:

```
Предсказанный класс: 1 (versicolor)

Вероятности классов:
  setosa: 0.0000 (0.00%)
  versicolor: 1.0000 (100.00%)
  virginica: 0.0000 (0.00%)
```

Модель возвращает индекс предсказанного класса (в данном случае — 1), который можно интерпретировать в зависимости от меток целевой переменной.

Априорные вероятности

Одним из преимуществ наивного байесовского подхода является возможность **задания априорных вероятностей** для целевых классов. Это позволяет учитывать заранее известные предположения о вероятности появления каждого класса.

В scikit-learn это реализовано через параметр `priors` в классе `GaussianNB`. Например, можно установить, что первый и второй классы встречаются с

вероятностью 0.25, а третий — с вероятностью 0.5:

```
# Создать модель с заданными априорными вероятностями
# Предположим, setosa встречается реже (25%), а virginica чаще (50%)
classifier_custom = GaussianNB(priors=[0.25, 0.25, 0.5])
model_custom = classifier_custom.fit(X_train, y_train)

# Сравнить предсказания
print("Вероятности (стандартные априорные):")
print(model.predict_proba(new_observation))

print("\nВероятности (пользовательские априорные):")
print(model_custom.predict_proba(new_observation))
```

Если параметр `priors` не указан, библиотека автоматически рассчитывает априорные вероятности на основе распределения данных в обучающем наборе.

Калибровка вероятностей

Следует помнить, что **сырые вероятности**, полученные с помощью метода `predict_proba()`, **не являются калиброванными**. Это означает, что если модель предсказывает вероятность 0.70, это не обязательно означает, что в 70% аналогичных случаев наблюдение действительно принадлежит этому классу.

Если требуется получить реалистичные вероятностные оценки (например, для систем поддержки принятия решений или оценки рисков), необходима **калибровка вероятностей** — этому посвящен раздел 5.2.

5.2 Калибровка предсказанных вероятностей наивного байесовского классификатора

Предсказанные вероятности классов имеют критическое значение во многих приложениях:

- **Медицинская диагностика**: "вероятность заболевания 85%" — основа для принятия решения о лечении
- **Кредитный скоринг**: вероятность дефолта определяет процентную ставку

- **Системы рекомендаций**: сортировка по уверенности модели
- **A/B тестирование**: принятие решений на основе порогов вероятности

Однако многие модели, включая наивный Байес, **выдают несколько необъективные ("некалиброванные") вероятности**. Например, модель может предсказывать вероятность 0.90, но в реальности только в 60% таких случаев наблюдение действительно принадлежит этому классу.

Калибровка — это процесс корректировки предсказанных вероятностей, чтобы они соответствовали реальным частотам. Хорошо откалиброванная модель должна удовлетворять условию:

> Среди всех предсказаний с вероятностью ~70%, примерно 70% должны быть правильными.

Для калибровки в **scikit-learn** используется класс `CalibratedClassifierCV`:

```
# Загрузить библиотеки
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.calibration import CalibratedClassifierCV
import numpy as np

# Загрузить данные
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42
)

# Создать базовую модель
classifier = GaussianNB()

# Создать откалиброванную модель с перекрестной проверкой
classifier_sigmoid = CalibratedClassifierCV(
    classifier,
    cv=5,  # 5-fold кросс-валидация
    method='sigmoid'  # метод калибровки Платта
)
# Обучить обе модели
```

```
classifier.fit(X_train, y_train)
classifier_sigmoid.fit(X_train, y_train)

# Создать новое наблюдение для демонстрации
new_observation = [[2.6, 2.6, 2.6, 0.4]]

# Сравнить сырые и откалиброванные вероятности
prob_raw = classifier.predict_proba(new_observation)
prob_calibrated = classifier_sigmoid.predict_proba(new_observation)

print("Сырые вероятности (GaussianNB):")
print(prob_raw)
print(f"  Setosa: {prob_raw[0][0]:.6f}")
print(f"  Versicolor: {prob_raw[0][1]:.6f}")
print(f"  Virginica: {prob_raw[0][2]:.6f}")

print("\nОткалиброванные вероятности:")
print(prob_calibrated)
print(f"  Setosa: {prob_calibrated[0][0]:.4f}")
print(f"  Versicolor: {prob_calibrated[0][1]:.4f}")
print(f"  Virginica: {prob_calibrated[0][2]:.4f}")
```

Результат:

```
Сырые вероятности (GaussianNB):
[[1.37182457e-05 9.99986079e-01 2.03233517e-07]]
  Setosa: 0.000014
  Versicolor: 0.999986
  Virginica: 0.000000

Откалиброванные вероятности:
[[0.05942731 0.8573782  0.08319449]]
  Setosa: 0.0594
  Versicolor: 0.8574
  Virginica: 0.0832
```

Интерпретация результатов:

- **Сырая модель** предсказывает вероятность класса Versicolor с уверенностью 99.99% — это явная переуверенность
- **Откалиброванная модель** дает более реалистичную оценку 85.74%, оставляя значительную вероятность для альтернативных классов

Методы калибровки в `CalibratedClassifierCV`:

- `sigmoid` — сигмоидная модель Платта, подходит для небольших и средних выборок;
- `isotonic` — изотоническая регрессия, непараметрическая, может переподгоняться на малых выборках.

Как работает CalibratedClassifierCV. Калибровка использует **k-fold кросс-валидацию**:

1. Данные делятся на k блоков (фолдов)
2. Для каждого фолда:
   - Модель обучается на k-1 блоках
   - Калибровка выполняется на оставшемся блоке
3. Итоговые вероятности — среднее по всем k моделям

Это предотвращает переобучение при калибровке на тех же данных, на которых обучалась основная модель.

5.3 Преимущества и недостатки наивного байесовского классификатора

Преимущества наивного байесовского классификатора:

- Простота и высокая скорость — алгоритм отличается лёгкостью реализации и быстрым обучением даже на больших объёмах данных.
- Эффективность при малом количестве обучающих примеров — метод хорошо работает, когда данных недостаточно для сложных моделей.
- Устойчивость к незначительным нарушениям предположений — даже при неполной независимости признаков классификатор часто сохраняет высокую точность.
- Хорошая масштабируемость — обучение и предсказание выполняются быстро благодаря вычислительной простоте и линейной зависимости от числа признаков.

Ограничения и недостатки наивного байесовского классификатора:

- Предположение независимости признаков — основное ограничение метода, которое редко выполняется на практике и может снижать точность на взаимозависимых данных.
- Плохая работа при сильной корреляции признаков — зависимые переменные нарушают байесовское предположение и искажают вероятностные оценки.
- Невозможность учёта взаимодействий между признаками — модель не способна выявлять сложные взаимосвязи или нелинейные зависимости.
- Чувствительность к неверным априорным вероятностям — если априорные распределения заданы некорректно, результаты классификации могут быть смещены.
- Ограниченная интерпретируемость вероятностей — выходные вероятности не всегда отражают истинные значения из-за упрощений в предположениях модели.